

Exam Solution for TDT4186

Spring 2025

1 Multi-Choice Questions

Automatic grading. Attention: Each question is worth 1.5 points.

1. Microkernel
2. A request made by a user program to the operating system kernel. **Explanation:** Functions in standard libraries are not necessarily system calls.
3. A program in execution
4. Disk size
5. fork()
6. To replace the current process with a new program
7. A process is a program in execution, while a thread is a subset of a process.
8. Global variables
9. SJF
10. Like FIFO
11. High context-switching overhead
12. Interactive processes
13. Memory management unit
14. It determines the physical addresses of a process during execution.
15. Splitting memory into variable-sized logical units
16. Best-Fit
17. 1,4
18. 1,3

19. Page number and offset
20. 14 bits
21. 27
22. A TLB miss occurs when the page table entry is not found in the TLB.
23. Round-Robin
24. 1024, 10 bits
25. 0,1,2
26. To signal threads to wake up when a specific condition is true
27. Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait
28. Reduces CPU overhead by allowing devices to transfer data directly to memory
29. By mapping device registers into the memory address space and accessing them with standard memory instructions
30. SSTF (Shortest Seek Time First)
31. 2,1
32. 13.17
33. Path, inode, file descriptor
34. Filename
35. File descriptor
36. 2
37. It contains metadata about the file system layout and status
38. Using a bitmap where each bit represents a block's availability

2 Multi-part questions

For multi-part questions, the automatic grading may fail to award some answers which are slightly different from the provided solutions or written in different phrases. Thus, it is worth noting that all wrong blanks should go through a scrutiny check. Answers that provide a close meaning can be awarded the point without deduction.

Process

The increment of the counter variable should be in the parent section and executed before calling `exec()` in the child.

- `fork()`
- `wait()`
- `exec()`
- D

Scheduling

- fairness
- Virtual runtime
- red-black tree
- P1
- 10

Paging

- 1FF
- 0F5
- 0DB
- EEf
- 12345EEf (the final level page directory stores the frame number)

Concurrency

Explanation: The ticket dispenser is a variation of the producer-consumer problem. The customer thread is the producer, while the `bank_staff` thread is the consumer. Thus, the initializations of the three semaphores are the same as in the producer-consumer problem. Both the `bank_staff` thread and the consumer threads access the waiting queue, while the `bank_staff` also has its own small queue — the service desk — which can be considered as a queue with a single empty slot.

1. The bank staff first needs to check whether there is a waiting customer, so it waits on the full semaphore.
2. It should update the seat information that is related to the empty semaphore.

3. Then, it can start to provide service to a customer by calling `sem_post` on the service semaphore.

This order should remain otherwise it may encounter some issues.

- If `sem_post(empty)` is called first, a new customer may join the queue even when no seat is actually available.
- If `sem_post(service)` is called first, the bank staff may attempt to serve a customer even when no customer is waiting in the queue.
- If `sem_post(service)` is called second, the system may not update seat availability in time. In a real scenario, when a customer is about to be served, they first leave their seat and go to the service desk, which immediately frees up a seat for another customer waiting to join the queue.

Answers:

- `semaphore empty =10;`
- `semaphore mutex =1;`
- `semaphore full =0;`

The following answers may be given in any order without penalty.

- `sem_wait(full)`
- `sem_post(empty)`
- `sem_post(service)`

I/O devices

As long as your answers convey the same meaning, you will receive the points.

- **Polling:** Polling is when the CPU repeatedly checks (polls) the status of an I/O device to see if it's ready for an operation **Interrupts:** Interrupts allow I/O devices to signal the CPU only when they need attention, so the CPU can do other work in the meantime.
- The application should use interrupts because polling wastes CPU time. The CPU is stuck checking the device instead of doing other tasks. The polling is inefficient for slow devices.